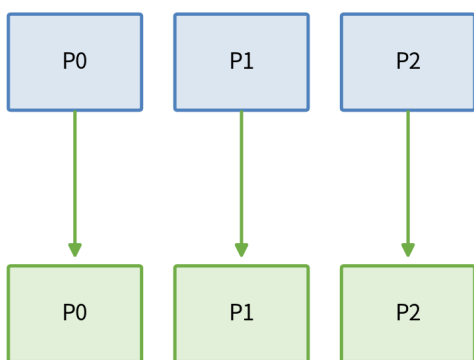


Spark 原理详解

第 13 讲：RDD 的核心机制

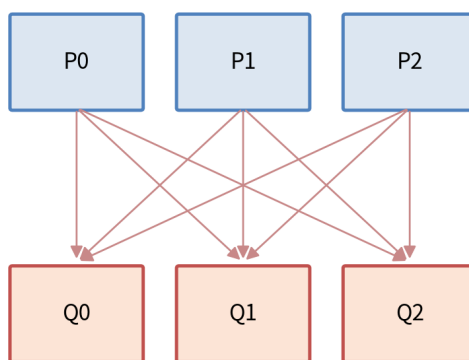
Narrow / Wide Dependency • Lineage • cache/persist • 容错重算

Narrow Dependency (窄依赖)



一个父分区通常只被少量子分区依赖
可流水线执行，通常不需要 Shuffle

Wide Dependency (宽依赖)



一个父分区的数据可能被多个子分区需要
需要重新分区和跨节点搬运，通常产生 Shuffle

本讲核心问题：Spark 为什么知道哪里需要 Shuffle？中间结果丢了为什么还能恢复？cache 为什么能加速，但又不能乱用？

学习目标：学完本讲后，你应该能看着一串 RDD 操作，判断依赖类型、Stage 边界、故障时重算范围，以及是否值得 cache/persist。

1. 先把 RDD 再认识一次：它不只是“一堆数据”

上一讲我们已经知道 RDD 是 Resilient Distributed Dataset（弹性分布式数据集）。现在要把这个定义再往底层推进一步：RDD 最重要的并不只是“数据分成多个 Partition”，还包括它知道这些 Partition 是怎样从上游数据计算出来的。

更准确的心智模型：RDD = 分区后的数据抽象 + 生成这些分区的计算规则 + 与上游 RDD 的依赖关系。

例如下面这段程序：

```
lines = sc.textFile("hdfs:///data/log.txt")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs:///out/wc")
```

Python 变量 lines、words、pairs、counts 看起来像“保存着数据”，但在真正执行之前，它们更像一张逐渐构建起来的计算说明书。Spark 会记录：counts 来自 pairs，pairs 来自 words，words 又来自 lines。这个关系链就是后面 Lineage 和容错的基础。

1.1 RDD 为什么必须不可变 (Immutable)

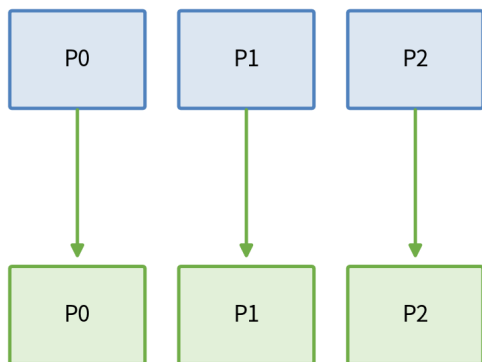
RDD 创建后不能在原地修改。map()、filter() 等操作会生成新的 RDD，而不是把旧 RDD 改掉。这个设计非常重要，因为只有上游数据和计算规则保持稳定，Spark 才能可靠地说：“某个分区丢了，我沿原来的计算链再算一次。”

类比：把 RDD 想成 Excel 中的公式链：A 列不直接被 B 列修改，而是 B 列用公式引用 A 列。只要原始数据和公式还在，某个结果格丢了就能重新计算。

2. Dependency: Spark 真正关心的是“分区之间怎么依赖”

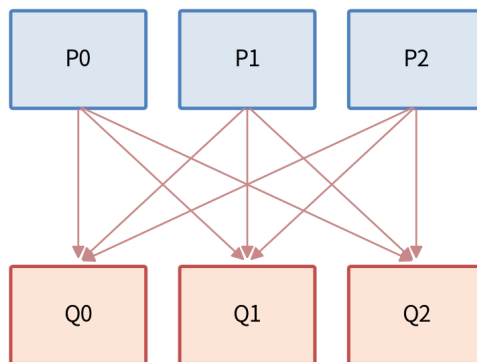
初学 Spark 时很容易只盯着函数名，例如 map、filter、reduceByKey。调度器真正关心的却是另一个问题：一个子 RDD 的某个 Partition，需要父 RDD 的哪些 Partition 才能算出来？

Narrow Dependency（窄依赖）



一个父分区通常只被少量子分区依赖
可流水线执行，通常不需要 Shuffle

Wide Dependency（宽依赖）



一个父分区的数据可能被多个子分区需要
需要重新分区和跨节点搬运，通常产生 Shuffle

图 1 Narrow Dependency 与 Wide Dependency 的直观区别

2.1 Narrow Dependency（窄依赖）

窄依赖中，一个父 Partition 的数据通常只会流向很少的子 Partition；对于最常见的 map、filter 这类一对一转换，一个子 Partition 基本只需要对应的那个父 Partition。

- map：P0 中的记录经过函数处理后，仍然形成下游某个对应分区的数据。
- filter：只是从当前分区删除不满足条件的记录，不需要找其他机器的数据。
- flatMap：一条记录可以变成多条，但通常仍在同一条分区计算链中完成。

为什么窄依赖便宜：可以把多个操作“串起来”在同一个 Task 中流水线执行，不必为了下一步先把全体数据通过网络重新洗牌。

3. Wide Dependency（宽依赖）：Shuffle 从这里出现

宽依赖的核心不是“函数比较复杂”，而是下游分区需要从多个上游分区收集数据。例如 reduceByKey() 要把所有相同 Key 的数据放到一起。假设 cat 同时出现在 P0、P1、P2，那么负责 cat 的下游 Partition 必须从多个父 Partition 把这些记录收集过来。

```
P0: (cat, 1) (dog, 1)
P1: (cat, 1) (bird, 1)
P2: (dog, 1) (cat, 1)

reduceByKey 之后：
cat 需要从 P0 + P1 + P2 收集数据
dog 需要从 P0 + P2 收集数据
```

这就无法只靠本地流水线完成，因为数据必须重新分区（repartition）并在 Executor/节点之间搬运。这个过程就是 Shuffle。

3.1 常见操作应该怎么判断

操作	典型依赖	通常 Shuffle?	直觉
map / filter / flatMap	窄依赖	否	每个分区自己算自己的
mapPartitions	窄依赖	否	对每个分区独立处理
reduceByKey	宽依赖	是	相同 Key 必须重新聚集
groupByKey	宽依赖	是	同 Key 的所有 Value 必须见面
distinct	通常宽依赖	通常是	重复值可能散落在多个分区
repartition	宽依赖	是	主动重新分布数据

判断法：不要死背函数。只问一句：下一步计算是否需要“别的分区上的数据”？如果需要大范围重新汇合，就很可能存在宽依赖和 Shuffle。

4. 为什么 Wide Dependency 往往形成 Stage 边界

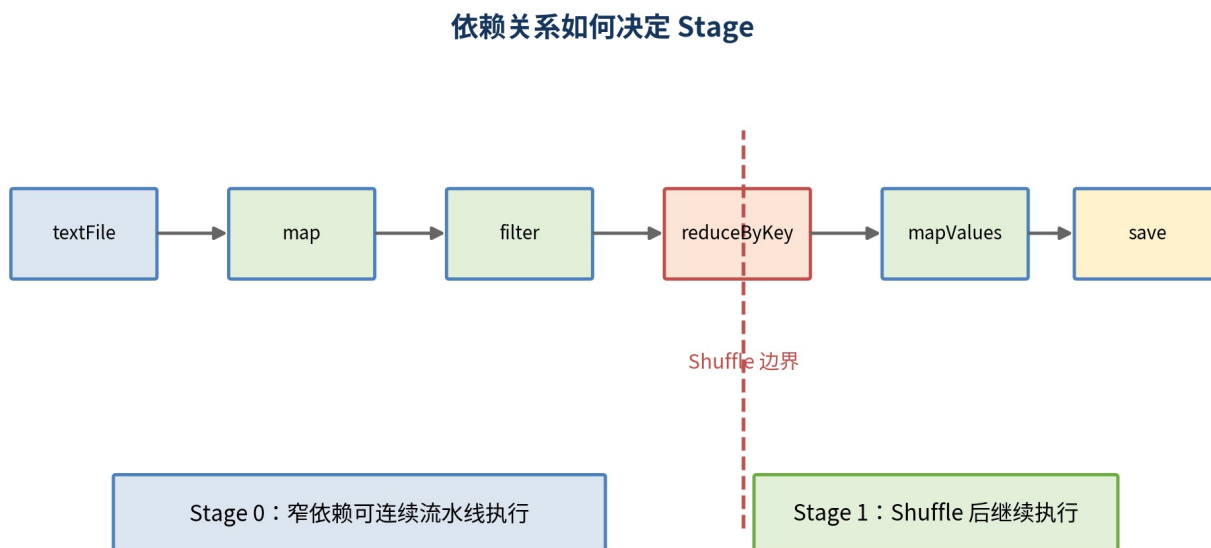


图 2 窄依赖可流水线；Shuffle 往往把 Job 切成多个 Stage

Spark 的 DAGScheduler 会沿着 RDD 依赖关系分析执行计划。连续的窄依赖通常可以合并到同一个 Stage 中；遇到 ShuffleDependency 时，必须先让上游 Stage 生成 Shuffle 数据，后面的 Stage 才能继续读取这些数据。

```
rdd = sc.textFile("hdfs:///data/orders.csv")      # Stage 0
x = rdd.map(parse)                                # Stage 0
x = x.filter(lambda r: r.amount > 0)               # Stage 0
x = x.map(lambda r: (r.city, r.amount))            # Stage 0
x = x.reduceByKey(lambda a, b: a + b)              # Shuffle 边界
x = x.mapValues(lambda v: round(v, 2))             # Stage 1
x.saveAsTextFile("hdfs:///out/city_sales")         # Action
```

这里并不是“每一个 Transformation 都产生一个 Stage”。相反，Spark 会尽量把能流水线执行的窄依赖操作放在同一个 Stage 内。

非常重要：Stage 的划分主要受 Shuffle 边界影响，而不是受代码行数影响。十个连续 `map()` 可能仍只有一个 Stage；一个 `reduceByKey()` 却可能把执行计划切开。

5. Lineage: Spark 为什么能 “记住怎么重新算”

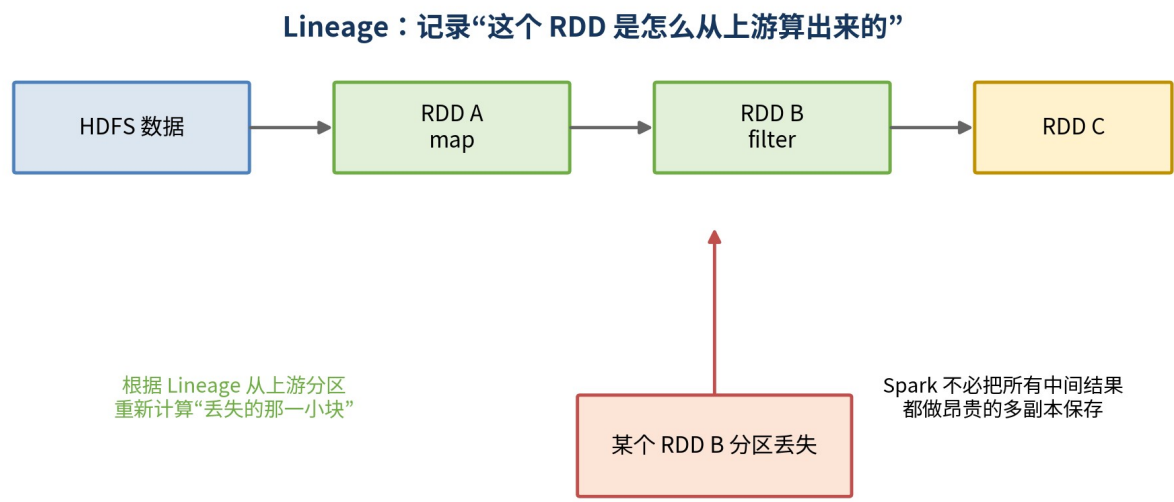


图 3 Lineage 让 Spark 可以沿上游重新计算丢失分区

Lineage（血缘/依赖链）记录一个 RDD 是如何由其他 RDD 经过 Transformation 得到的。它不是给人看的“日志”，而是 Spark 容错模型的重要结构。

假设 RDD B 有 100 个 Partition，其中第 37 个 Partition 所在 Executor 故障，且该分区没有可靠副本。Spark 通常不需要把整个 RDD B 从头算一遍，而是根据 Lineage 找到生成 B[37] 所需的上游分区，再重新执行那部分计算。

5.1 这和 HDFS 的副本容错有什么区别？

系统	核心对象	典型容错思想
HDFS	原始/持久化数据块	提前保存多个副本；机器坏了仍可从其他副本读取
Spark RDD	可重新计算的中间数据	保存计算血缘；分区丢失时沿 Lineage 重算

这体现了一个很漂亮的系统思想：有些数据“重新算比保存很多副本更划算”。Spark 不是完全不保存中间结果，而是在“重算成本”和“存储成本”之间做设计权衡。

6. 容错重算并不意味着“任何失败都很便宜”

Lineage 让恢复成为可能，但恢复成本取决于依赖关系。如果丢失的是窄依赖链中的某个分区，通常只需重算少量上游分区；如果涉及 Shuffle 输出丢失，可能需要重新运行相应的 Shuffle Map Task。依赖链特别长时，恢复也会越来越昂贵。

错误理解：“RDD 有 Lineage，所以失败几乎没有成本。”——不对。Lineage 解决的是可恢复性，不保证恢复一定便宜。

6.1 一个具体例子

```
A = sc.textFile(...)
B = A.map(f1)
C = B.filter(f2)
D = C.map(f3)
E = D.reduceByKey(add)
F = E.mapValues(f4)
```

如果 C 的一个 Partition 丢了，通常可以从 A 的对应输入分区重新执行 f1、f2；但如果 E 之后依赖的某个 Shuffle 数据丢失，Spark 可能需要回到产生该 Shuffle 输出的上游任务重新执行。

因此 Spark 性能分析里，“依赖链”“Shuffle”“缓存”“失败恢复”并不是四个互不相关的概念，它们是同一个执行模型的不同侧面。

6.2 什么时候 Lineage 会变成负担？

- 迭代次数非常多，形成很长的依赖链。
- 上游计算非常昂贵，失败后重算成本高。
- 一个基础 RDD 被后续很多 Action 反复使用，每次都从头计算。

这就引出了 cache / persist 和 checkpoint。

7. cache(): 解决的是“同一份中间结果被反复使用”

不缓存：每个 Action 都可能沿 Lineage 重算

缓存：复用昂贵的中间结果

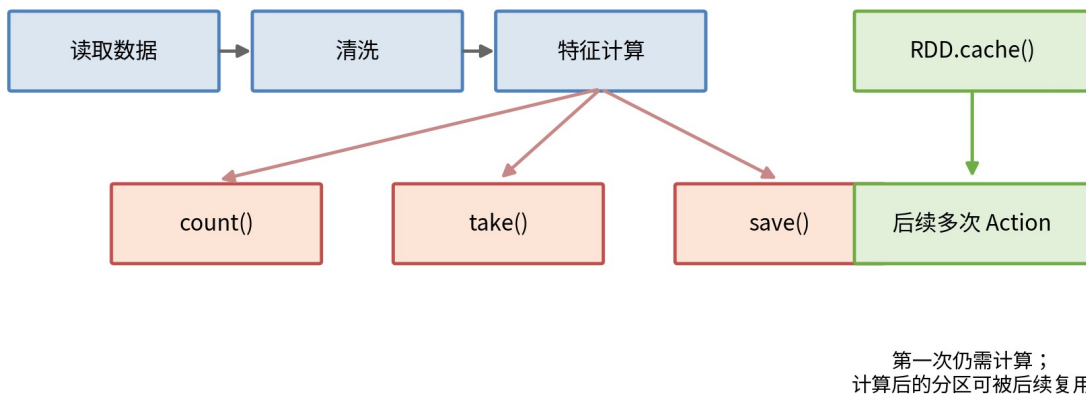


图 4 cache 的核心价值：避免后续 Action 反复重算昂贵的上游链

Spark 的 Transformation 是惰性的。如果一个清洗后的 RDD 被多个 Action 使用，而你没有缓存，那么不同 Action 可能再次沿 Lineage 执行相同的清洗和特征计算。

未缓存

```
clean = raw.map(parse).filter(valid)

print(clean.count())
print(clean.take(10))
clean.saveAsTextFile(...)
```

如果 clean 的计算代价很高，可以：

缓存后

```
clean = raw.map(parse).filter(valid).cache()

print(clean.count())      # 第一次 Action：真正计算，并把分区缓存下来
print(clean.take(10))     # 后续 Action：尽量直接复用缓存
clean.saveAsTextFile(...)
```

关键点：cache() 本身也是惰性的。调用 cache() 只是“标记这个 RDD 值得保存”；直到第一个 Action 到来，RDD 才真正被计算并逐步缓存。

8. persist(): 缓存不是只有“放内存”一种方案

persist(): 在“速度、内存、CPU、磁盘”之间做权衡



并不是“级别越高越好”，而是根据数据大小、复用次数和资源压力选择

图 5 常见持久化思路：内存、内存+磁盘、仅磁盘

cache() 是方便用法；persist() 允许你明确选择 StorageLevel。具体 API 默认值可能随 RDD/DataFrame 和 Spark 版本不同，因此工程中如果存储策略重要，最好显式选择，而不是只凭记忆默认值。

```
from pyspark import StorageLevel

rdd.persist(StorageLevel.MEMORY_AND_DISK)
# 第一个 Action 后开始物化缓存
rdd.count()

# 不再需要时释放
rdd.unpersist()
```

8.1 cache 什么时候值得用？

场景	建议	原因
同一个昂贵 RDD 会被多个 Action 重复使用	很适合	避免反复计算共同上游
迭代算法每轮都反复访问同一训练数据	很适合	复用频率高
RDD 只使用一次	通常不值得	缓存本身也有写入/管理成本
数据远大于可用内存	谨慎	可能频繁淘汰、落盘，收益下降
缓存了大量再也不用的数据	不应该	占用 Executor 内存，挤压执行空间

原则：不要把 cache 当成“加速按钮”。先问：这份中间结果以后会不会被重复用？重算它贵不贵？资源够不够？

9. cache/persist 与 checkpoint：目标不同

cache/persist 与 checkpoint 不是一回事



实际工程里二者可以同时出现：cache 解决“重复算”，checkpoint 解决“依赖链过长”

图 6 cache 主要解决复用；checkpoint 主要解决过长依赖链

checkpoint 会把 RDD 的数据物化到可靠存储，并截断此前的 Lineage。之后恢复这个 RDD 时，不需要重新穿过完整的早期依赖链。

```
sc.setCheckpointDir("hdfs:///user/hadoop/spark-checkpoint")

rdd = ...
rdd.checkpoint()
rdd.count()    # 通过 Action 触发真正计算与 checkpoint
```

9.1 两者不要混为一谈

问题	cache / persist	checkpoint
主要目的	后续复用更快	截断 Lineage、提高长链恢复可控性
是否保留原 Lineage	通常保留	成功 checkpoint 后可切断前序依赖
典型存储	内存和/或磁盘	可靠文件系统（集群中通常 HDFS 等）
典型场景	反复使用同一中间结果	超长 Lineage、复杂迭代计算

实际工程里二者可以同时出现。例如某个迭代算法既希望频繁复用中间 RDD，又希望每隔若干轮截断一次很长的 Lineage。

10. 把四个概念串起来：一次完整推理

假设我们分析 500 GB 用户行为日志，程序如下：

```
raw = sc.textFile("hdfs:///logs/2026/")
clean = raw.map(parse).filter(valid)
user_action = clean.map(lambda x: (x.user_id, 1))
user_count = user_action.reduceByKey(lambda a, b: a + b).cache()

active = user_count.filter(lambda x: x[1] >= 100).count()
top10 = user_count.takeOrdered(10, key=lambda x: -x[1])
```

1. `textFile` → `map` → `filter` → `map`：这些通常是窄依赖，可以在同一个上游 Stage 中流水线执行。
2. `reduceByKey`：相同 `user_id` 可能散布在多个 Partition，因此产生宽依赖和 Shuffle，形成新的 Stage 边界。
3. `user_count.cache()`：只做标记；第一次 `count()` 触发计算后，`user_count` 的分区开始被缓存。
4. 第二个 Action `takeOrdered()` 可以复用 `user_count`，而不是再次读取并清洗 500 GB 原始日志。
5. 如果某个已经缓存的分区后来丢失，Spark 仍可根据 Lineage 从上游重新生成它。

你现在应该形成的链条：Dependency 决定 Shuffle → Shuffle 影响 Stage → Lineage 记录如何计算 → 丢失分区可重算 → `cache/persist` 减少重复重算。

11. 上机实验：亲眼观察 Partition、Lineage 与 Cache

下面实验不依赖大数据，目标是“观察 Spark 行为”。在已有 Spark 环境中进入 PySpark：

```
pyspark
```

然后逐行运行：

```
rdd = sc.parallelize(range(1, 1000001), 4)
print(rdd.getNumPartitions())

x = rdd.map(lambda n: n * 2)
y = x.filter(lambda n: n % 3 == 0)

print(y.toDebugString())

y.cache()
print(y.count())
print(y.take(5))

# 查看缓存状态
print(y.getStorageLevel())

# 释放缓存
y.unpersist()
```

11.1 你应该观察什么

- `getNumPartitions()`：确认 RDD 的并行分区数量。
- `toDebugString()`：观察 RDD 的依赖链表示，不必一开始看懂所有内部类名。
- 第一次 `count()`：触发真正计算，同时物化 cache。
- 第二次 `take(5)`：理论上可以复用已经缓存的分区。
- Spark Web UI 的 Storage 页面：观察缓存了多少 Partition、占用多少空间。
- Stages 页面：再尝试加入 `reduceByKey`，观察是否出现新的 Shuffle Stage。

实验目的：不是比速度，而是建立“代码 - Lineage - Stage - Cache”之间的对应关系。

12. 初学者最容易犯的 10 个错误

误区	正确理解
1. RDD 就是一份已经存在内存里的数据	RDD 更重要的是分区数据抽象和计算依赖；未被 Action 触发时可能尚未真正计算。
2. 每个 Transformation 都会产生一个 Stage	通常 Shuffle 边界才是 Stage 切分的关键。
3. map 一定比 reduceByKey 快	关键不在函数名字，而在数据量、函数成本和是否 Shuffle。
4. Wide Dependency 就是“函数复杂”	宽依赖描述的是分区间的数据依赖关系。
5. Lineage 是日志文件	它是计算依赖图的一部分，是容错重算依据。
6. cache() 一调用就把数据塞进内存	cache 也是惰性的，需 Action 触发计算。
7. 所有 RDD 都应该 cache	只用一次的 RDD 往往不值得缓存。
8. cache 后绝不会重算	缓存可能被淘汰或丢失，仍可沿 Lineage 重算。
9. checkpoint 就是 cache 的高级版本	目标不同：一个侧重复用，一个侧重截断 Lineage。
10. Spark 容错完全靠缓存	容错的基础是 Lineage；缓存主要是性能手段。

13. 练习题：先自己判断，再看答案

题 1：下面哪一步最可能产生 Shuffle？为什么？

```
a = rdd.map(f)
b = a.filter(g)
c = b.map(lambda x: (x.city, 1))
d = c.reduceByKey(lambda x, y: x + y)
```

题 2：clean RDD 只会被 saveAsTextFile() 使用一次，是否应该 cache？

题 3：一个 RDD 的第 12 个分区丢失，Spark 是否一定要重新计算整个 RDD？

题 4：一个程序有 20 个 map/filter，但没有 Shuffle，是否一定会有 20 个 Stage？

题 5：某个迭代算法的 Lineage 已经延伸数百步，即使有 cache，为什么还可能考虑 checkpoint？

参考答案

6. d = reduceByKey 最可能产生 Shuffle，因为同一个 city 的记录可能来自多个上游 Partition，需要重新聚集。
7. 通常没必要。只使用一次时，缓存写入和占用资源可能没有收益。
8. 不一定。窄依赖情况下常常只需要根据 Lineage 重算生成该分区所需的上游分区；具体范围取决于依赖和 Shuffle。
9. 不一定。连续窄依赖可以被流水线放在同一个 Stage 中。
10. cache 解决的是复用效率，并不天然截断 Lineage；依赖链太长时，checkpoint 可以把恢复起点前移。

本讲一句话总结：Spark 的 RDD 核心不是“把数据放内存”，而是用 Partition + Dependency + Lineage 描述分布式计算，再用 Shuffle、Stage、重算和缓存控制执行成本。

14. 后续课程还剩 7 讲

为了避免课程无限扩展，我们把 Spark 这一阶段固定在第 20 讲收束。完成本讲以后，剩余路线如下：

讲次	主题	解决的核心问题
第 14 讲	Spark Shuffle 与分区	reduceByKey/groupByKey、Partitioner、Shuffle 文件、数据倾斜为什么慢
第 15 讲	DataFrame 与 Spark SQL	为什么现代 Spark 更多使用结构化 API；Schema、Column、SQL
第 16 讲	Catalyst 与执行计划	Logical Plan、Physical Plan、explain、Join 与优化器到底做什么
第 17 讲	Spark on YARN 与资源模型	Driver、Executor、Container、CPU/Memory 怎么分配与运行
第 18 讲	Spark 性能优化	Partition 数、缓存、Broadcast Join、倾斜、小文件、内存与 GC
第 19 讲	完整 PySpark 项目实战	HDFS → 清洗 → 聚合 → SQL → 输出 → Web UI → 排错闭环
第 20 讲	MapReduce + Spark 综合总结	把 Hadoop/Spark 的底层概念统一起来，并自然过渡到 Flink

这 7 讲结束后，你不需要继续深挖大量冷门 Spark API。对于“大数据处理技术”课程而言，已经足以建立从 MapReduce 到 Spark 的完整心智模型，并具备继续学习 Flink 的基础。

本讲最终记忆图

请把下面 6 个箭头记住：



下一讲：Spark Shuffle 与分区。它会把你前面学过的 Hadoop Shuffle 与 Spark Shuffle 放在一起对照，这是理解 Spark 性能问题最重要的一讲之一。